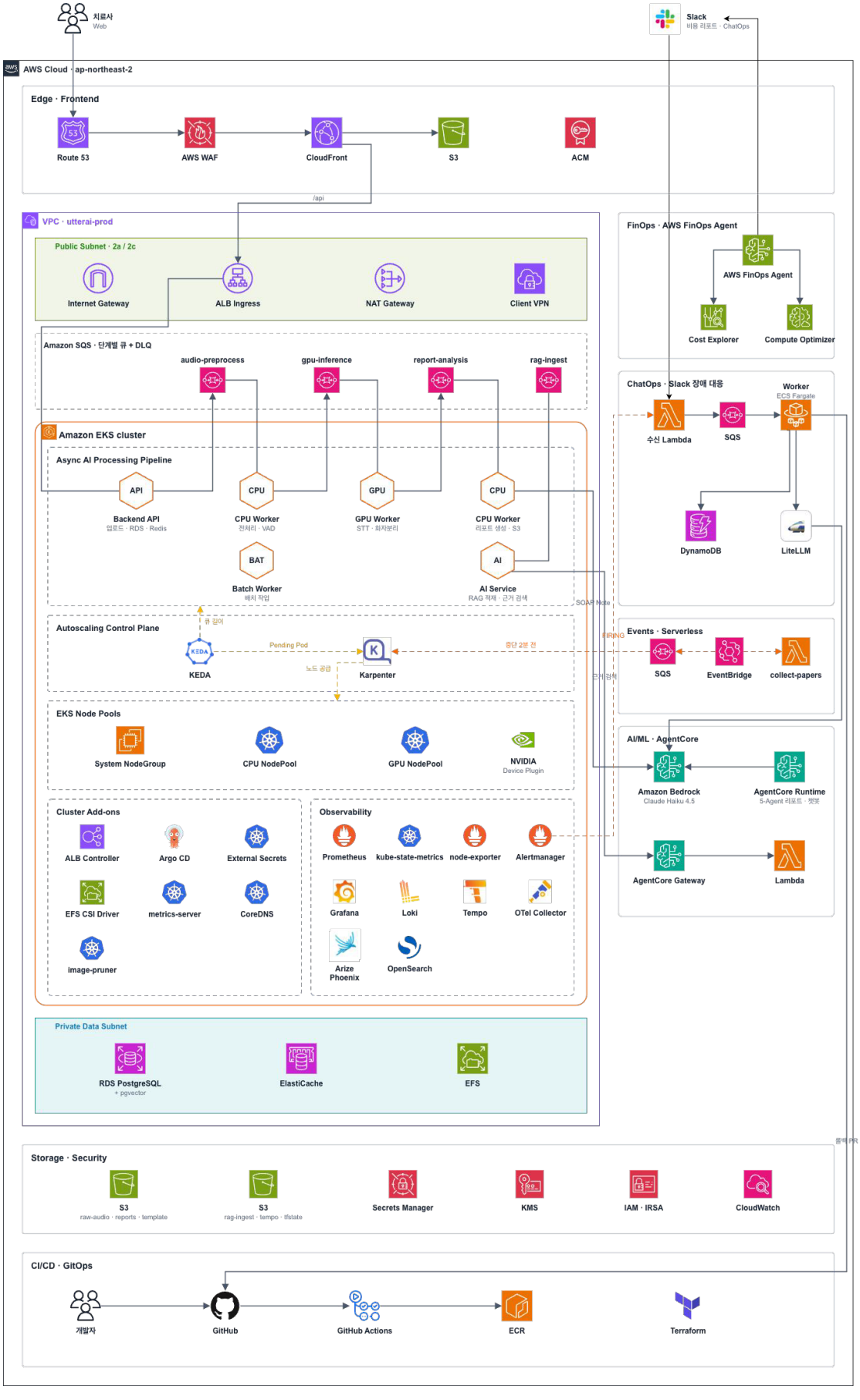
UtterAI

2026.05 - 2026.07

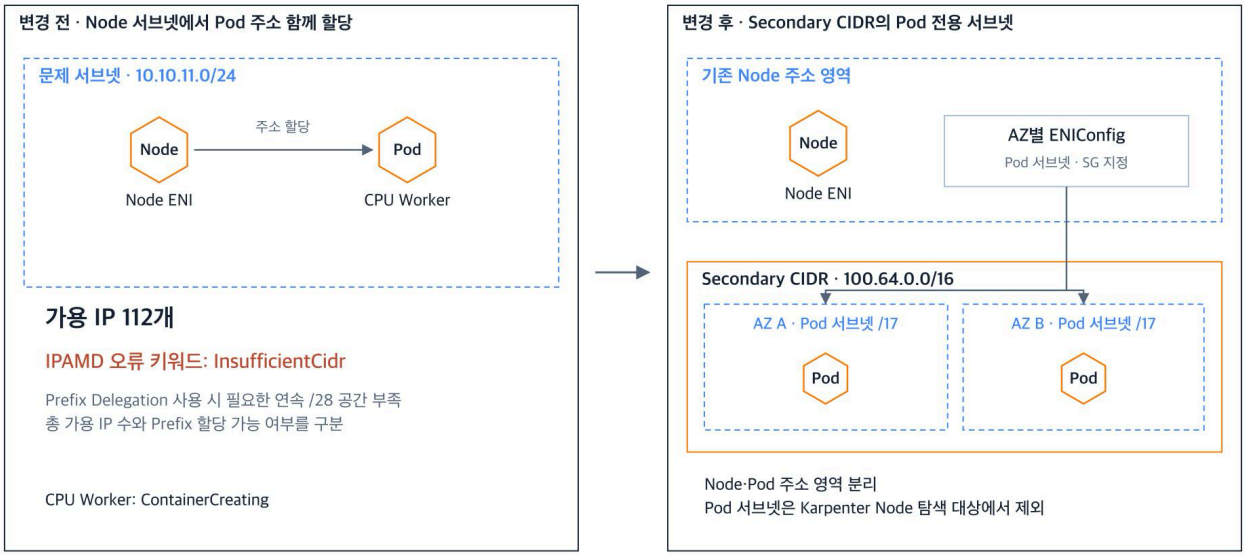
Demo GitHub

언어재활사가 치료 세션 녹음을 업로드하면, 발화자를 구분한 전사문과 언어지표를 분석하고 SOAP 형식의 치료 기록 초안을 생성하는 서비스입니다. 치료사는 분석 결과와 초안을 검토·수정하여 치료 기록을 완성할 수 있습니다.



1. IPAMD 로그로 EKS Pod IP 할당 실패 원인을 규명하고 Custom Networking으로 Node·Pod 주소 영역 분리

| Prefix Delegation 서브넷 단편화로 인한 Pod IP 할당 실패를 Secondary CIDR 기반 Pod 전용 주소 영역으로 해결



문제·원인

- CPU Worker의 ContainerCreating 상태 지속 및 Pod IP 할당 실패
- 문제 서브넷에 가용 IP 112개가 남아 있어 전체 IP 소진만으로 설명하기 어려운 장애
- Prefix Delegation은 정렬된 /28 블록 단위로 IP를 할당하는데, VPC Endpoint ENI와 Node별 예비 Prefix가 서브넷을 단편화시켜 할당 가능한 /28 블록이 남지 않음

해결과정

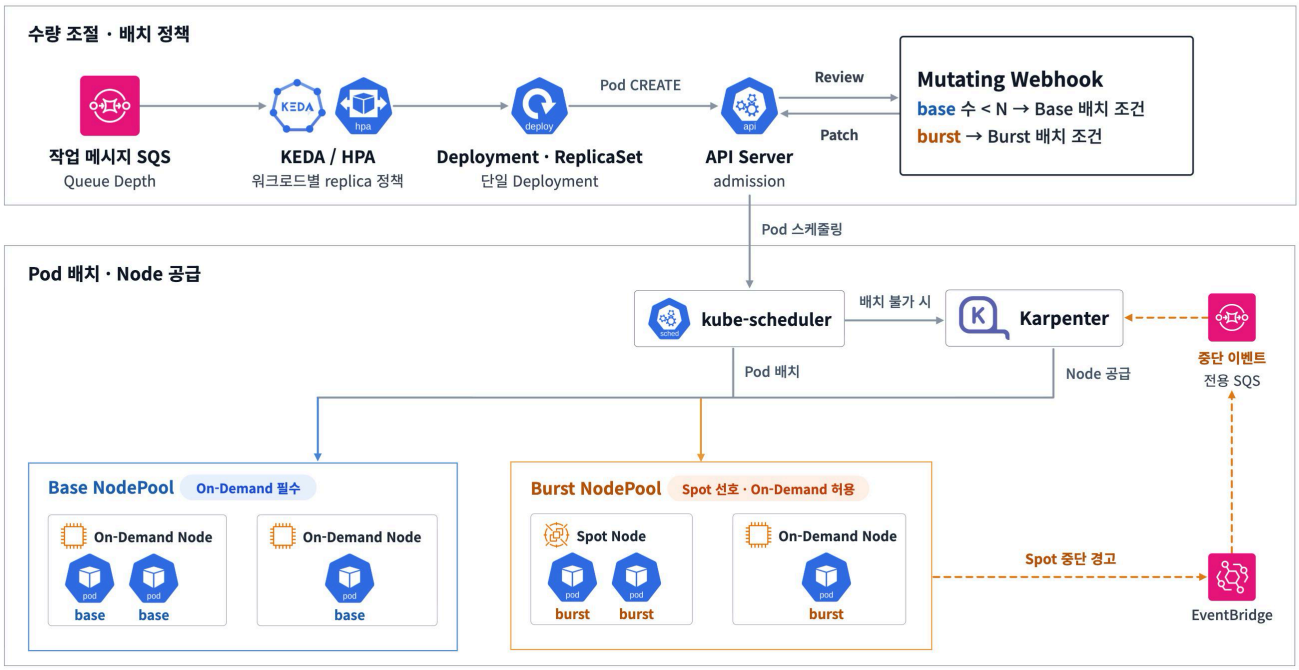
- Pod 이벤트·Node ENI 상태를 확인한 뒤 VPC CNI·IPAMD 로그를 추적해 InsufficientCidr 오류 확인
- Secondary CIDR(100.64.0.0/16)에 AZ별 Pod 전용 서브넷을 만들고, Custom Networking으로 Node·Pod 주소 영역 분리
- AZ별 ENIConfig를 연결하고, Node가 Pod 대역 IP를 받지 않도록 Karpenter의 Node 서브넷 탐색 대상에서 Pod 서브넷 제외
- DNS 조회 실패를 양쪽 Node tcpdump로 추적해 보안 그룹 단계의 차단으로 좁히고, Pod용 ENI와 CoreDNS Node간 상호 허용 규칙 추가
- VPC Endpoint 보안 그룹이 기존 VPC 대역만 허용해 SQS 연결이 실패하던 문제를 Pod 대역 추가로 해결

결과

- 새 Pod가 100.64.x.x 대역에서 IP를 받아 ContainerCreating 해소
- Pod 할당 가능 주소를 AZ당 약 250개에서 약 3.2만 개로 확장하고, Node 서브넷은 Node당 IP 1개만 쓰는 구조로 바꿔 단편화 재발 방지

2. SQS Depth 기반 KEDA·Karpenter 오토스케일링으로 워크로드별 자원 조절 및 EC2 비용 절감

| SQS 대기량에 따라 Worker를 늘리고, 배치 정책으로 Base / Burst를 나눈 뒤 Karpenter가 조건에 맞는 Node를 공급



문제·원인

- CPU 사용률 70% 기준 HPA가 GPU 연산 부하와 SQS 대기량 증가를 감지하지 못해 Scale-out이 안됨
- HPA의 최소 replica가 1개라 요청이 없어도 GPU·Batch가 상시 실행되어 유휴 비용 발생
- CA에서는 Spot을 섞으려면 구매 방식별로 노드 그룹을 따로 구성해야 함
- Spot 도입 후 On-Demand에 남길 기본 용량 기준이 없어, 사용자 요청을 처리할 기본 용량까지 Spot 회수 위험
- GPU를 0개까지 축소하자, 업무 시작 후 첫 요청이 Node 공급과 모델 기동을 기다리며 약 5분 딜레이

해결과정

1. SQS 작업량에 따른 Worker·Node Scale out

- KEDA로 확장 기준을 CPU에서 SQS로 바꾸고, visible-in-flight 메시지를 합산해 처리 중인 작업이 지표에서 누락되지 않도록 구성
- 워크로드별 목표 큐 길이로 Pod당 처리 메시지 수를 정하고, min/max replica로 자원 사용 범위 설정
- 기존 Node에 여유가 없어 Pod가 Pending 상태가 되면, Karpenter가 Pod의 CPU·GPU 요구사항에 맞는 Node를 공급

2. Scale-to-zero와 Karpenter 기반 Node 비용 최적화

- NodePool 하나에 Spot·다중 인스턴스를 함께 지정할 수 있는 Karpenter 채택
- 워크로드별 NodePool로 인스턴스 후보와 Node limits를 따로 관리하고, CPU·GPU Worker는 빈 Node만 정리해 실행 중인 작업 보호
- Batch/GPU는 min replica를 0으로 두어 요청이 없을 때는 Pod와 Node를 모두 정리
  (GPU는 업무 시간 예외, 3번 참고)

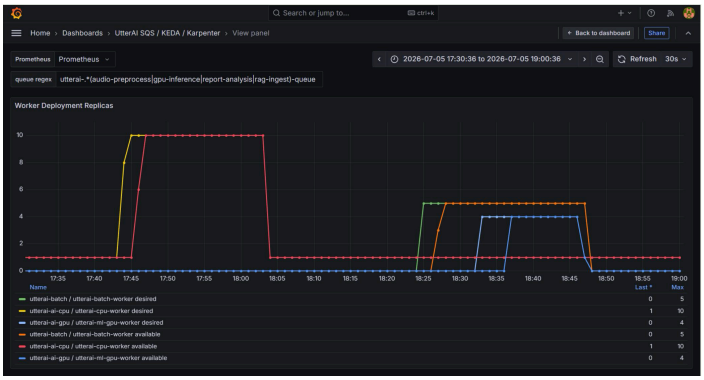
3. 기본 용량과 추가 용량의 배치 정책 분리

- **배치분리**
  - base는 On-Demand 전용 NodePool, burst는 Spot을 선호하되 공급이 부족하면 On-Demand도 허용하는 NodePool에 배치
- **단일 Deployment에 Mutating Webhook 적용**
  - Pod 수는 KEDA가 정하고 각 Pod의 base/burst 배치는 webhook이 분류하도록 역할 분리
  - 여러 웹훅 복제본이 동시에 요청을 처리해도 base가 중복 배정되지 않도록, ConfigMap 예약과 resourceVersion 기반 충돌 재시도 적용
  - 원본 Pod 템플릿을 On-Demand로 두고 'failurePolicy: Ignore'를 적용해, 웹훅 장애 중에도 Pod 생성 가능하도록 구성
- **GPU warm base**
  - 평일 08:30~20:00에는 KEDA Cron으로 1개를 미리 띄워 유지하고, 업무 외에는 0개로 축소
- **축소와 Spot 중단 대응**
  - pod-deletion-cost를 차등 설정해, 같은 ReplicaSet 안에서는 burst가 먼저 삭제되도록 유도
  - EventBridge와 중단 이벤트 전용 SQS로 Karpenter가 Spot 중단 경고를 받도록 구성
  - Worker 종료 시 새 작업 수신을 중단 후 진행 중 작업을 마무리하고, 강제 종료된 작업은 SQS visibility가 만료되면 다시 수신

결과

- 오토스케일링·Spot·인스턴스 구성 조정을 통해 EC2 Compute **일비용 $28.73 → $12.02, 약 58% 절감**
- Grafana의 Worker별 desired·available replica 비교를 통한 실제 확장·축소 동작 확인

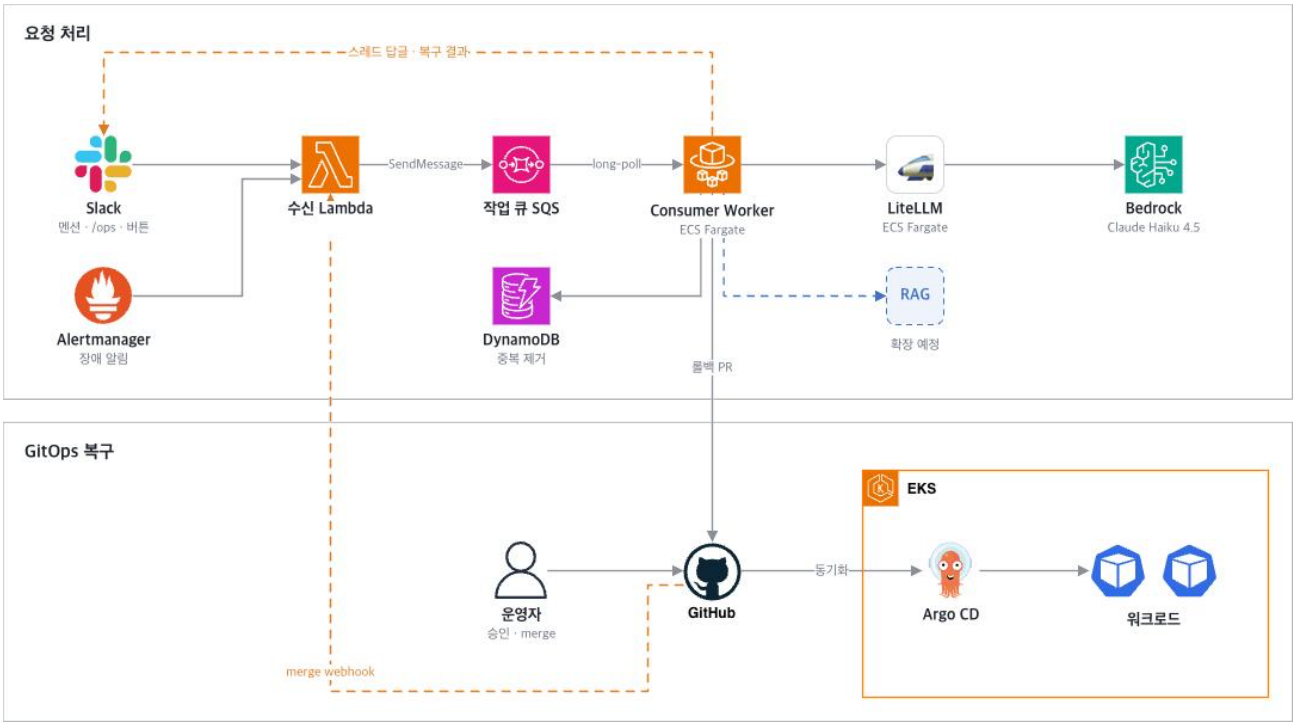
  Worker 목표 및 가용 replica 변화



- 업무 시간에만 warm GPU 1개를 유지해 기본 유지 시간 **주 168시간 → 57.5시간 감소**

3. LiteLLM과 Slack 기반 ChatOps로 장애 조사 및 운영자 승인 후 복구 절차 통합

| 장애 근거를 모아 롤백 PR을 제안하고, 승인·반영 후 복구 확인까지 Slack에서 처리



문제·원인

- 음성 분석·리포트 생성이 단계별 큐와 워커로 나뉘어 있어, 장애 지점과 원인을 찾으려면 단계마다 큐 상태·Pod 이벤트·최근 배포 이력을 따로 확인해야 해 대응이 오래 걸림
- 롤백한 뒤에도 새 버전이 반영되어 Pod가 정상 동작하는지 따로 확인하고 그 결과를 팀에 다시 알려야 해, 대응을 마치기까지 시간이 추가로 걸림
- 채팅 봇으로 옮겨도 Slack은 3초 안에 응답받지 못하면 요청을 다시 보내는데 LLM 답변은 그보다 오래 걸려, 모델 호출과 메시지가 중복됨
- 수시로 들어오는 질의응답과 장애 근거 요약은 응답 속도·입력량·허용 비용이 달라, 하나의 모델 사용 정책으로 묶기 어려움

해결과정

1. Slack ChatOps 도입으로 알림 하나에 장애 원인이 되는 배포 정보 표시

- 알림·배포 이벤트·운영자 명령을 하나의 큐로 받아 처리하고, 실패 Pod의 이미지 태그를 배포 저장소 커밋과 연결해 어느 배포가 원인인지 알림에 바로 표시
- 배포가 원인이면 규칙에 따라 롤백 PR 생성 버튼을 제공

2. 복구 확인을 사람 대신 merge 이벤트가 시작

- 롤백 PR의 merge 이벤트를 받아 Argo CD 동기화, 새 이미지 Pod Ready, 재시작 횟수를 차례로 확인해 장애 스레드에 결과 보고
- Rolling update 중에는 기존 Pod 때문에 Ready 2/2로 정상처럼 보여, Ready 수 대신 새 이미지 Pod 기준으로 판정

3. **Slack 요청 접수와 처리를 비동기로 분리해 Slack 재전달에도 모델 호출과 답글은 한 번만 나가도록 처리**
    - Lambda는 서명 검증 후 SQS에 넣고 즉시 응답. 워커는 요청별 처리 상태와 생성 결과를 DynamoDB에 저장해 재전달 시 끝난 추론을 재사용하고, 전송 결과가 불확실한 답글은 다시 보내지 않고 격리
4. **LiteLLM 게이트웨이로 용도별 모델·호출량·비용 분리**
    - 앱은 fast·quality 별칭만 부르고, 서비스별 키에 허용 모델·분당 호출 수·예산을 지정해 질의응답은 빠른 모델, 장애 요약은 큰 모델로 고정
    - 호출 한도는 Redis로 프록시 간에 공유하고, 게이트웨이 장애처럼 모델에 닿지 않은 실패는 재시도 횟수에서 제외

결과

- 장애 알림이 오면 6초 안에 원인이 된 배포 커밋을 Slack에 표시해, 원인을 찾는 데 걸리던 시간을 2시간 5분에서 크게 단축
- 롤백 PR이 merge되면 33초 만에 복구 여부를 확인해 장애 스레드에 자동 보고
- 봇이 혼자 merge하려는 시도는 HTTP 405로 차단해, 운영자가 승인한 롤백만 반영
- Slack이 같은 요청을 3번 다시 보내도 모델 호출과 답글은 1번만 실행
- 질의응답 모델을 Haiku로 분리해 응답 시간 6.6초 → 2.4초로 감소하고, 호출 비용은 1/3로 감소